

# 基于分组逃逸的 2.5D 芯粒布线算法

于鉴轩

(计算机科学与技术系，学号：2025310688，手机号：18801329227)

陈佳华

(计算机科学与技术系，学号：2025210768，手机号：13979078709)

**摘要：**后摩尔时代，2.5D 异质集成芯粒方案是提升算力的关键路径，基板布线作为核心环节，需解决有限空间内海量信号的无冲突路径规划这一复杂组合优化问题。为此，本文提出一种基于分组逃逸的 2.5D 芯粒布线算法。该算法首先通过改进的 K-Means 聚类算法实现线网分类，以斜率初划分与引脚位置精划分相结合的方式生成线网分割集，为并行布线奠定基础；随后通过层分配将总线布线转化为图着色问题，采用回溯法或 DSATUR 算法求解，在避免总线冲突的同时最小化布线层数；接着基于引脚分布规律构建逃逸位置，利用二维 A\* 算法完成逃逸布线，通过 52 种模式化路径实现高效区域布线，对布线失败的引脚采用三维 A\* 算法重新布线；最后引入多线程并行优化提升求解效率。在 C2IO1 测例上的实验表明，该算法实现 100% 线网布通率，布线总层数为 3，求解总时间约 180 秒，多线程配置较单线程效率提升 47.8%，在保证布线质量的同时显著提升了求解效率。算法的核心优势在于通过分组缩小搜索空间、模式化布线降低复杂度，其瓶颈集中在 A\* 相关布线阶段，未来可通过优化数据结构进一步提升并行效果。

**关键词：**2.5D 芯粒；K-Means 聚类算法；层分配算法；A\* 算法；逃逸布线；模式布线

## 1. 引言

### 1.1 背景

基于 2.5D 异质集成技术的芯粒（chiplet）方案是后摩尔时代继续提升算力的一个重要路径。典型的 2.5D 结构如图 X 所示，基板（interposer），为芯粒之间数据交换的高速通信平台。它是一个集成了多层高密度金属互连网络的基板，通常采用硅基或者扇出的方式实现。与 PCB 以及有机基板相比，基板能够提供数量级上的互连密度提升，从而实现前所未有的高带宽与低延迟性能。布线是设计基板的关键环节，在基板上为所有跨芯粒的信号网络规划物理连接路径的过程。而本项目的核心任务，正是面向基板这一先进封装技术中的核心部件，设计并实现一套高效、高质量的自动化布线算法，以解决在有限空间内连接海量信号的复杂工程问题。

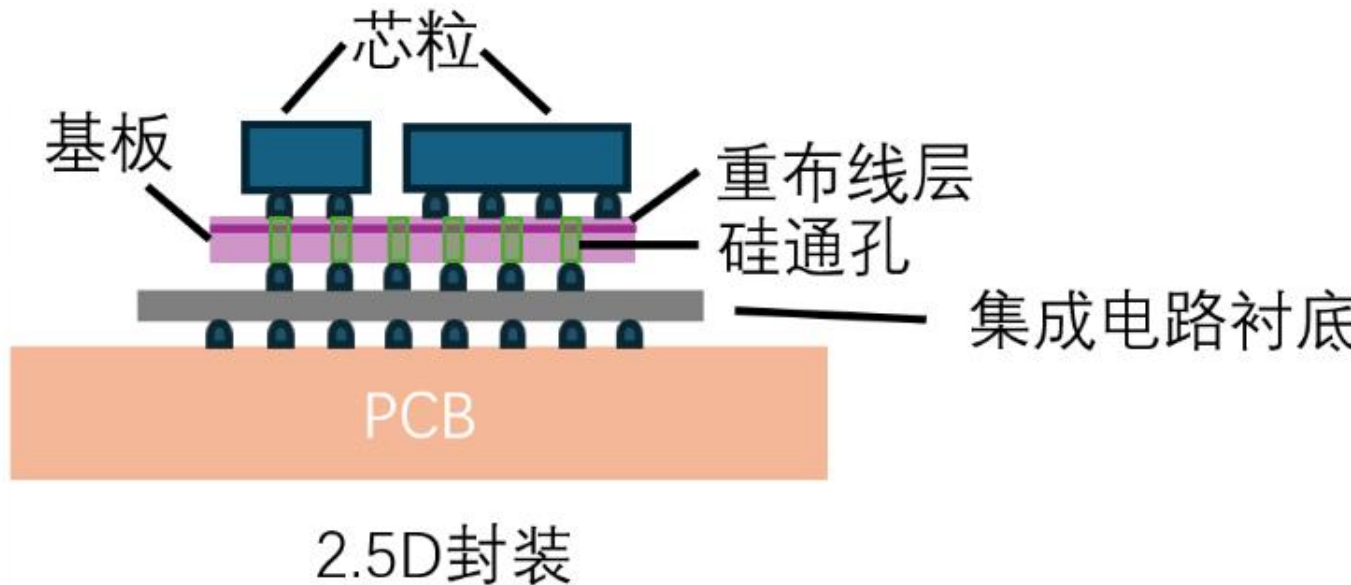


图 1. 2.5D 典型结构

如图 1，在 2.5D 集成系统中，芯粒通过其底部的微凸点（micro-bump）与下方的基板进行高密度的电气与物理连接。在整个 2.5D 模块的底部，则分布着尺寸和间距远大于微凸点的凸点（bump），主要的实现方式是 C4 凸点（controlled collapse chip connection bump, C4 bump）它们负责将基板作为一个整体，连接到下一级的封装基板或 PCB。本项目中的布线就是将完成微凸点到微凸点、微凸点到 C4 凸点之间的连接。在硅基板中（silicon interposer），硅靠近微凸点一面的金属线另一面的金属线通过硅通孔（through silicon via, TSV）连接，而基于

扇出工艺的基板由于没有硅，无需 TSV。本项目简化这一结构，不考虑 TSV。

## 1.2 问题

布线问题抽象为离散化的基于网格（grid）的数学模型。在此模型中，基板的每一个布线层都被划分为一个均匀的二维网格。网格是此次赛题走线的最小单位。不超过一个微凸点或凸点会落在一个网格中，作为布线路径的起点和终点。一条布线路径被定义为一系列在水平、垂直方向上相邻的网格序列，在 135 度布线场景中（或称 X routing），斜向相邻的网格也会连接，如图 2 所示。每个网格代表了单位长度的布线资源，在同一层中，一个网格在同一时间只能被一条布线路径占用。当一条路径需要从一个布线层切换至相邻的另一层时，必须使用一个垂直的通孔（Via），该通孔将同时占用其所在位置（同一  $x,y$  坐标）的两个相邻层中的网格资源。通过这种方式，复杂的物理设计问题被严谨地转化为一个在三维网格图上，为大规模网络集合求解无冲突路径的组合优化问题。当采用全局布线+局部布线的两阶段布线方法时，那么多个网格会集合成更大的布线单元（G-Cell）。

算法的性能可以评估为在满足按照网格布线、线网没有出现短路/断路、线网之间没有交叉重叠情况下的布线长度（用布线路径占用网格的数目表示）、布线层数（全部引脚分布在 Top 层和 Bottom 层，中间可以任意添加布线层）、以及算法求解效率。本工作的目标是设计一个在满足所有基本要求情况下，实现尽可能短的耗时、少的布线资源的布线算法。

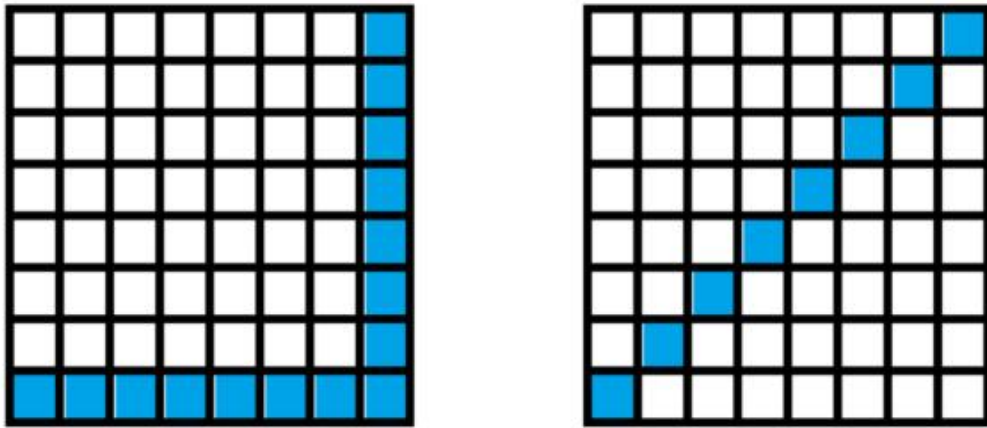


图 2. 布线路径示例

## 2. 基于分组逃逸的 2.5D 芯粒布线算法

针对上述问题，本文提出了基于分组逃逸的 2.5D 芯粒布线算法。本文提出算法的主要流程如图 3 所示：

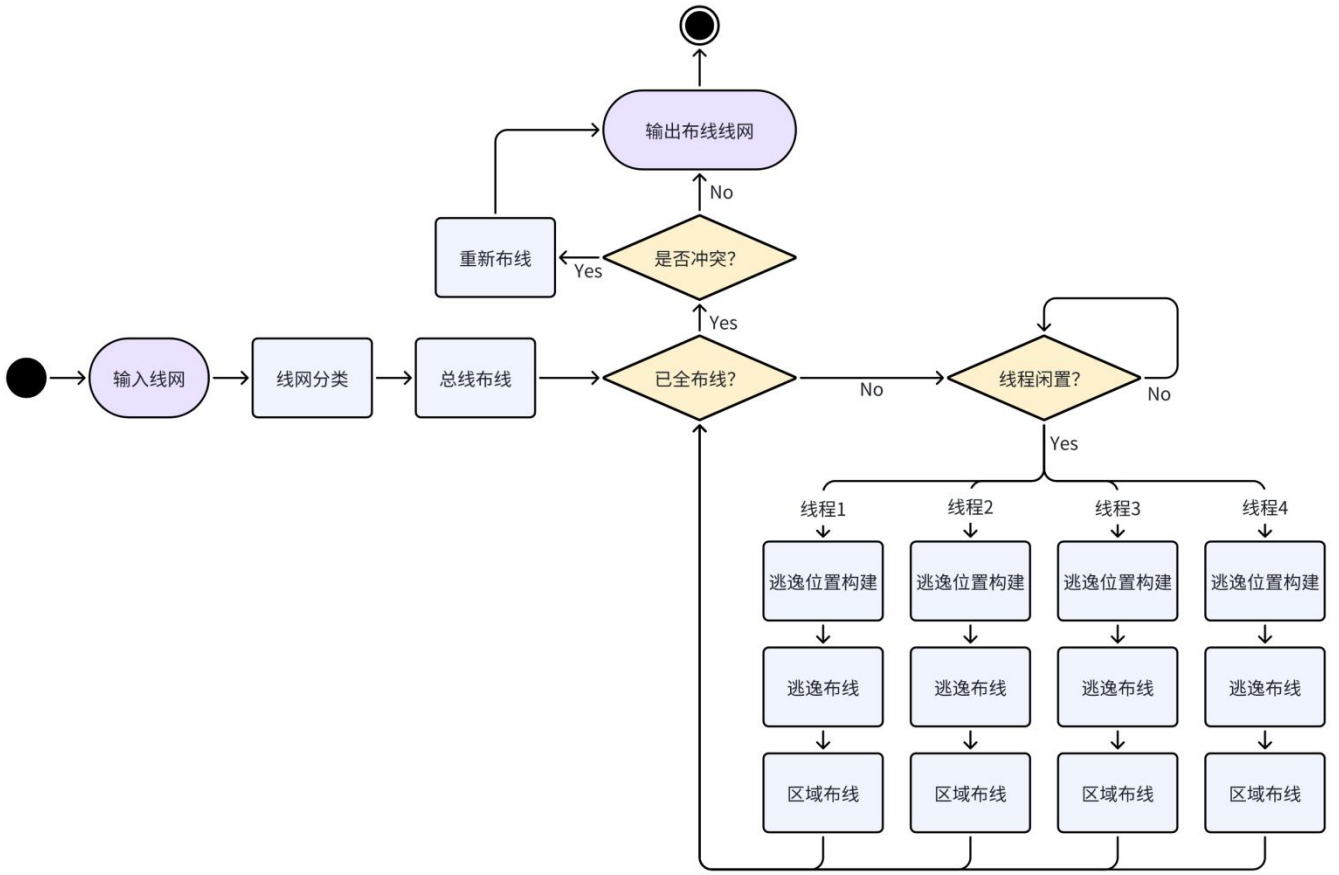


图 3. 算法流程

对于布线测例，考虑到 2.5D 芯粒布线问题中引脚分布的特性，本文提出的算法先使用 K-Means 聚类算法进行线网分类，每一类内的全部线网构成一个总线，每个总线由两个引脚集合构成，两个集合中的引脚一一对应。然后，进行基于层分配的总线布线，为每个总线分配布线资源，目标为，在不同总线之间布线资源不冲突的基础上，使得布线需要的层数最小。然后在各总线之间进行多线程并行处理，先为每个总线中的每个引脚集合构建逃逸位置，然后先后进行基于二维 A\*算法的逃逸布线和基于模式的区域布线。最后，对于布线失败的引脚，再进行基于三维 A\*算法的重新布线。接下来，本文将逐一介绍上述布线阶段。

## 2.1 基于K-Means聚类算法的线网分类

通过对待布线线网引脚分布进行初步观察，如图 4，可以发现这些线网往往聚集分布。如果对聚集起来的线网分别布线，一方面可以将全局的布线可行域缩小到只覆盖此类线网的可行域，从而减少搜索空间加快布线速度；另一方面，不同线网之间所占用的空间也会不同，这样方便并行布线。在这样的动机下，本工作引入了适用于线网分类的 K-Means 算法。定义一类线网满足类内的引脚出发点和结束点要聚集，同时，在引脚直连 line 段的斜率上要相近。这样得到的分类结果让后续的布线具有两个性质：组内的线连接时不会有交叉、通过预分配，分组后线网所占用的空间可以避免重叠，从而便于多线程的并行布线。

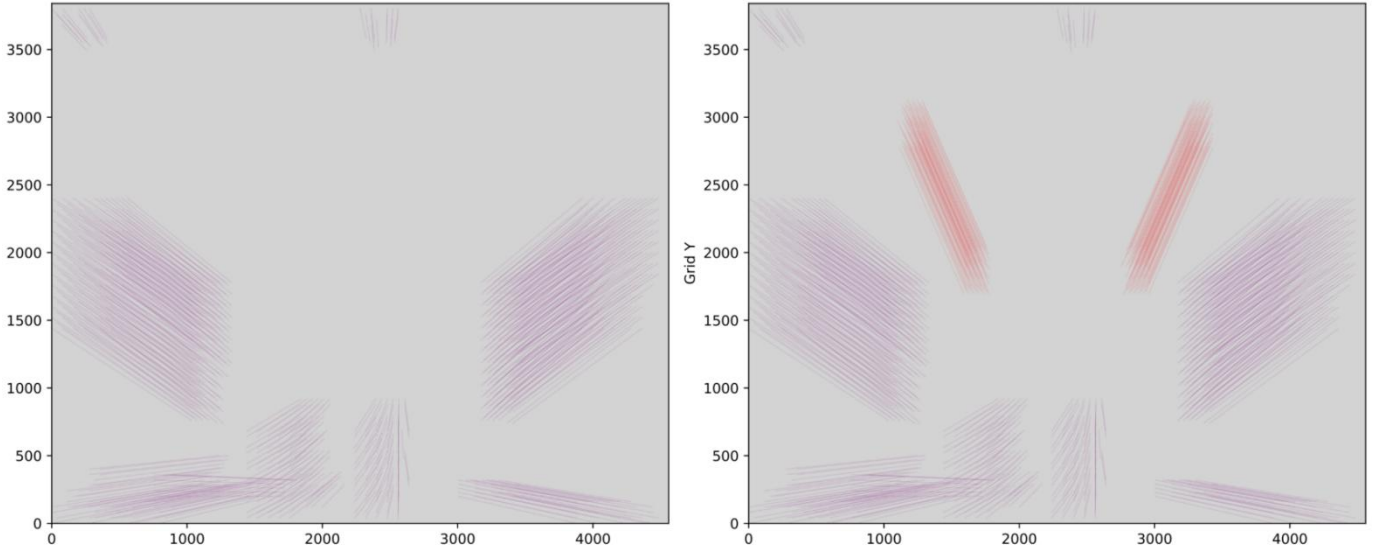


图 4. 引脚与直连线分布

算法的流程如算法 1 所示，核心通过斜率初划分 + 引脚位置精划分实现线网的合理分组，先对输入线网做预处理，提取引脚距离矩阵、最远引脚序号、引脚斜率三类核心特征；再将二维空间切分为 720 个角度区间，按斜率将线网归至对应区间，遍历区间并通过候选队列累计空区间数量，当空区间数超过斜率阈值  $z$  时完成一次初分组，得到若干初始线网组；最后对每个初始组，从初始化子组数和参考集开始，选取参考集线网在最远引脚序号中 TopK 共现的线网扩展参考集、增加子组数，以参考集为初始值执行 Kmeans 聚类得到子组，若子组间最近引脚距离小于组内平均最近引脚距离则终止迭代，否则重复上述引脚位置划分步骤，最终所有初始组的子组共同构成算法输出的线网分割集。

算法 1. 基于 K-Means 的线网分类算法

输入：线网  $N$ ，斜率阈值  $z$   
 输出：线网的分割集  $G1, G2, \dots, GN$   
 流程：  
 1. 预处理出线网引脚距离矩阵  $D$ 、最远引脚序号  $L$ 、线两个引脚的斜率  $K$   
 2. 斜率初划分线网组，得到初始组  $g1, g2, \dots, gp$ ，流程如下：  
   a. 将二维空间切分为 720 个角度区间，每个线依据斜率放置在对应该区间，维护一个候选队列  $Q$ ，累计空区间数目  $m$   
   b. 从小到大遍历区间，当区间不为空时，将区间内所有线网放入到  $Q$  中，当区间为空时， $m += 1$ ，当  $m$  大于  $z$  时，将  $Q$  中所有线网划分为一类，清空  $Q$   
 3. 按照引脚位置划分线网组，流程如下：  
   a. 遍历每个初始组  $gi, i = 1, 2, \dots, p$ ，对每个组维护一个子组数  $n$ （初始为 1），参考集  $R$ （初始为第一个线网）  
   b. 选取参考集  $R$  中每个元素在  $L$  中 TopK 共现的线网（假设为  $nf$ ），子组划分数目  $n$  加一， $nf$  加入到参考集  $R$  中  
   c. 执行 Kmeans 算法，组数目为  $n$ ，初始元素为  $R$ ，得到子组划分  $gij, j = 1, 2, \dots, n$ ，当任意两个子组之间的最近引脚距离小于组内的平均最近引脚距离时，结束循环；否则，回到 a 继续循环  
 4. 将  $gij, i = 1, 2, \dots, p, j = 1, 2, \dots, n$ ，作为输出的分割集  $G1, G2, \dots, GN$

算法在 C2IO1 用例上得到的分类结果图 5 所示，不同的线网颜色表示不同的类别，从肉眼结果的观测上，算法有效地按照预期分割了不同的线网，从而为后续资源分配、多线程布线提供了极大的便利。值得注意的是，本工作提出的 K-Means 改版算法只需要输入斜率阈值一个超参数即可，原始 K-Means 算法需要输入组数以及组内初始中心位置，这对于庞大的线网数据来说是不可确定的，以至于原始 K-Means 算法结果不佳。

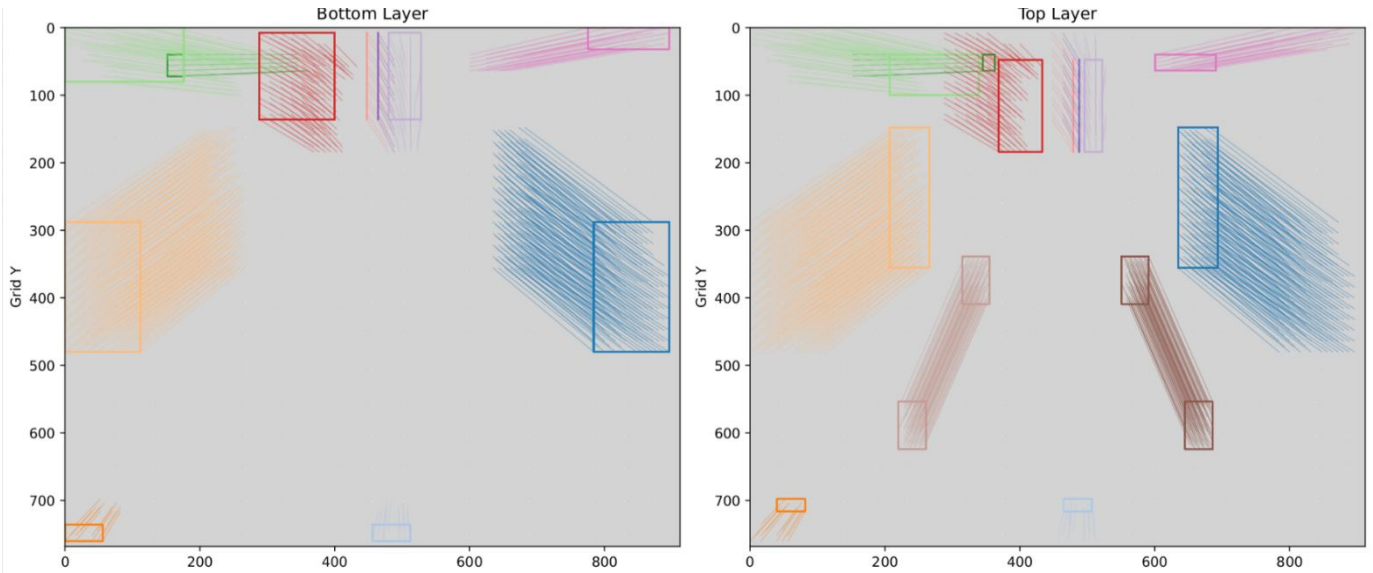


图 5. 基于线网分类的 K-Means 算法结果

2.2 基于层分配的总线布线

在线网分类结束后，每一类线网构成一个总线，每个总线包含两个引脚集合。接下来，本工作使用层分配的方式为每个总线分配布线资源，在不同总线之间布线资源不冲突的基础上，使得布线需要的层数尽可能小。在进行总线布线后，不同总线之间的布线则不再相互干扰，从而简化了后续布线流程。具体步骤为：首先确定总线占用的布线资源，然后确定不同总线之间的布线资源冲突，最后构建并求解图着色问题。

2.2.1 确定总线占用的布线资源

在本工作中，对于每一个总线，使用一个切去了 0 或 1 或 2 个等腰直角三角形的凸多边形，来表示其占用的布线资源。具体而言，根据总线中的两个引脚集合所形成的矩形区域进行分类讨论，每种情况分别对应的总线占用的布线资源如图 6 所示：

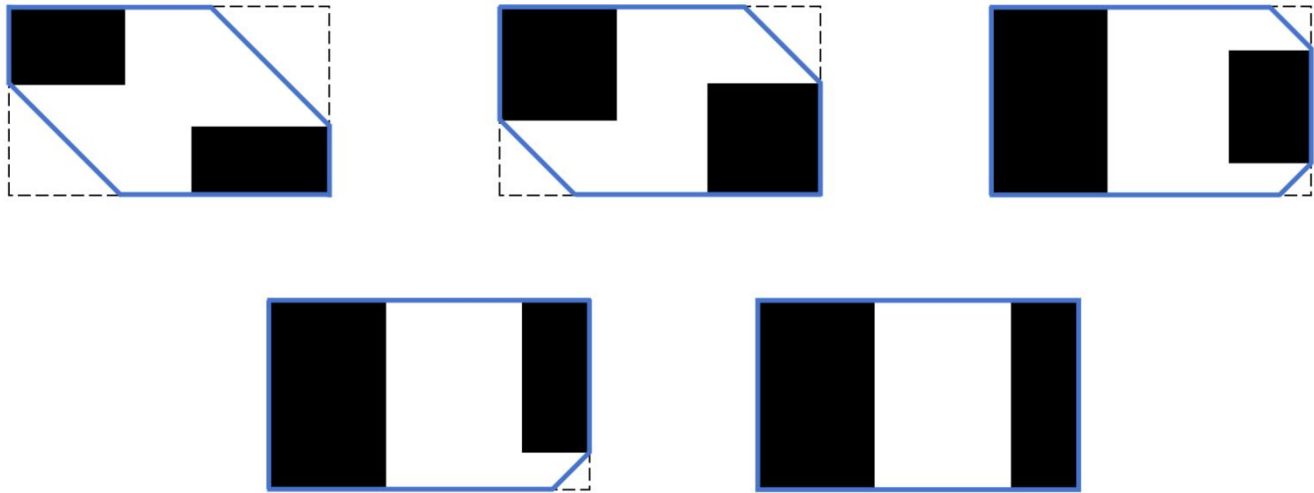


图 6. 总线占用布线资源

在上图中，5 个子图表示总线占用布线资源的 5 种不同情况。在每一个子图中，两个黑色矩形代表总线中的两个引脚集合所形成的矩形区域，蓝色边框的凸多边形表示在该种情况下总线占用的布线资源。考虑到本工作中 135 度布线的场景，上述表示策略是具有合理性的。

2.2.2 确定不同总线之间的布线资源冲突

确定不同总线之间的布线资源是否冲突，等价于确定不同总线对应的凸多边形是否相交。根据分离轴定理，如果两个凸多边形没有重叠，那么存在一个轴，使得物体在该轴上的投影不会重叠，如图 7 所示。在实际应用中，受益于凸多边形的性质，只需要依次在两个凸多边形的每条边的法向量上做投影即可[1]。

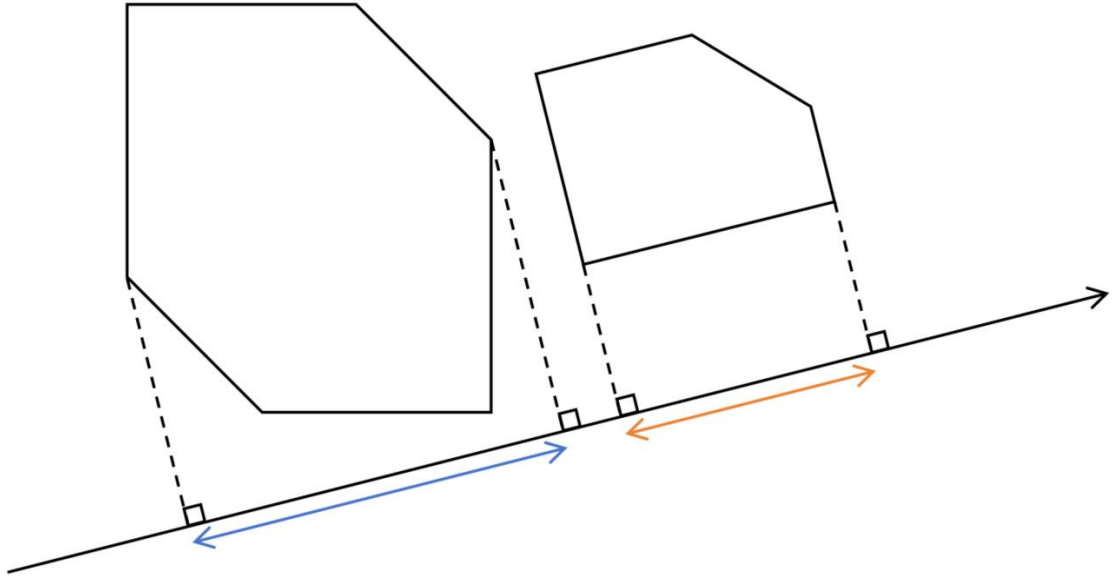


图 7. 分离轴定理

综上所述，本工作根据分离轴定理，遍历两个总线对应的两个凸多边形的每条边的法向量，从而确定两个总线之间的布线资源是否冲突。

### 2.2.3 构建并求解图着色问题

在确定不同总线之间的布线资源是否冲突后，本工作对全部总线进行布线资源分配，具体而言，为每一个总线分配布线层，使得布线资源冲突的总线不在同一布线层内。

用顶点表示每个总线，用顶点之间是否存在邻边表示总线之间是否冲突，用为顶点染上不同颜色表示为总线分配不同层，则总线布线资源的层分配可以形式化为一个图着色问题：对图中全部顶点分配颜色，使得相邻顶点着色不同。

为了求解上述图着色问题，本工作采用如下的求解办法：如果图中顶点数目较少，则使用基于回溯法的最优化求解[2]；如果图中顶点数目较多，则使用基于贪心算法的启发式求解，具体而言，本文使用了根据顶点饱和度从大到小进行排序，对顶点进行依次着色的 DSATUR 算法进行着色[3]。通过求解图着色问题，本工作实现了对总线布线资源的分配。

## 2.3 基于引脚分布模式的逃逸位置构建

对于每个总线内部的两个引脚集合，本文提出的算法分别在其对应的矩形边缘处，根据 2.5D 芯粒的引脚分布规律，为每个引脚构造逃逸位置，目标为尽量减小不同引脚对应“引脚-逃逸位置”连接之间的拓扑相交，从而减小后续的逃逸布线的难度。

在 2.5D 芯粒布线问题中，引脚分布具有一定规律：在同一总线内，不同线网对应的“引脚-引脚”连接之间的拓扑相交天然较少。如图 8 所示：

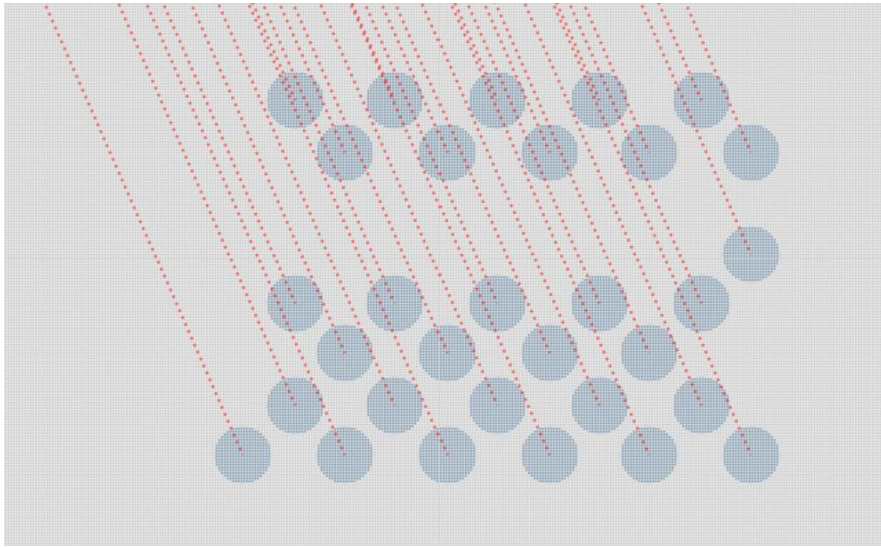


图 8. 2.5D 芯粒布线问题中的引脚分布

在上图中，蓝色圆形表示引脚，红色虚线表示“引脚-引脚”连接形成的线段，从上图中可以看到，全部红色虚线整体上的走势一致，可见“引脚-引脚”连接之间的拓扑相交较少。

本文提出的算法考虑利用上述特性，对于每个引脚，在“引脚-引脚”连接形成的线段与该总线对应矩形边缘的交点附近构建逃逸位置，保证同一总线中，同一线网的两个引脚在两个引脚集合中对应的逃逸位置在矩形边缘上的环绕顺序是一致的，且逃逸位置彼此不重合。

#### 2.4 基于A\*算法的逃逸布线和重新布线

在为每个引脚构建逃逸位置后，按照逃逸位置在矩形边缘上的环绕顺序，对每个引脚依次进行逃逸布线，从引脚所在位置出发，在引脚集合对应的矩形区域范围内，使用二维的A\*算法布线到该引脚对应的逃逸位置[4]。

在A\*算法的过程中，为了保证探索位置的合法性，本工作面向布线网格构建包含虚拟结点的连通图，构建过程如图9所示：

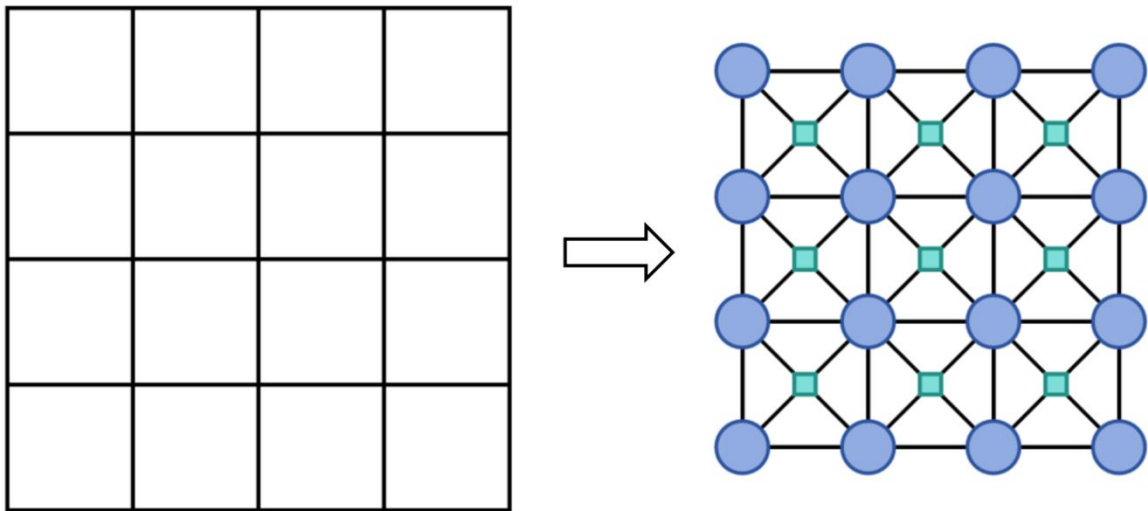


图 9. 包含虚拟结点的连通图构建过程

在上图中，左图表示布线网格，右图表示包含虚拟结点的连通图，其中，蓝色圆形表示布线位置，青色方形表示虚拟结点，蓝色圆形和青色方形均为连通图中的顶点，黑色线段表示顶点之间的邻边。上述连通图能够辅助算法在 135 度布线的场景下能够正确布线，不出现布线交叉重叠。同时，在布线过程中，为了保证探索位置的合法性，算法需要通过分类讨论，避免布线轨迹中出现 45 度锐角转折。

在本工作中使用的面向 2.5D 芯粒逃逸布线的 A\*算法中，应用当前探索位置在不考虑布线路径交叉的情况下到达终点所能花费的最小网格数目作为启发式函数，这种启发式函数设计实际上表示了在当前规定的布线长度路径成本的衡量下，当前探索位置与终点之间的“距离”。

如果应用上述的基于二维 A\*算法的逃逸布线算法无法将部分引脚连接至其对应的逃逸位置，则记录全部布

线失败的引脚，在区域布线阶段结束之后，再在整个布线区域范围内，依次使用三维 A\*算法进行重新布线。

## 2.5 基于模式的区域布线

在之前步骤中，引脚逃逸点的构建性质保证了对任意一对线网逃逸点平行地连线不会交叉。在只能竖直、水平、对角布线的要求下，直接计算对应的拐点即可。计算需要考虑到要连接的两个逃逸点坐标以及逃逸出边对应的组边界，于是共有 52 种情况，第一类为无拐点情况，如图 10 所示，这种情况发生在程序判断两个逃逸点可以直接连接时。图所展示的是斜率为反对角情况，还有其余 3 种分别为水平、竖直、对角情况，于是无拐点情况共有 4 种可能。

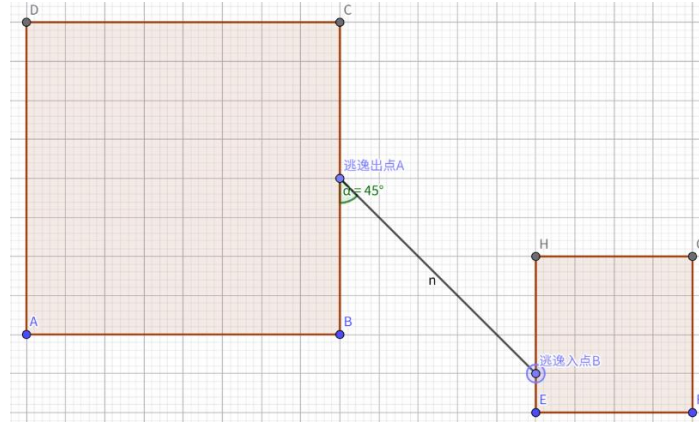


图 10: 无拐点情况

第二类为有一个拐点的情况，如图 11 所示。逃逸点所在的逃逸边界不管是出边还是入边都会对拐点位置的计算产生符号上的差异，分别考虑逃逸点在边界的所有情况（上、下、左、右）以及是否是出发情况（出、入）进行讨论，于是共有  $4 \times 4 \times 2 = 32$  种可能性。图 X 的拐点坐标公式为  $I(A.x + A.y - B.y, A.y)$

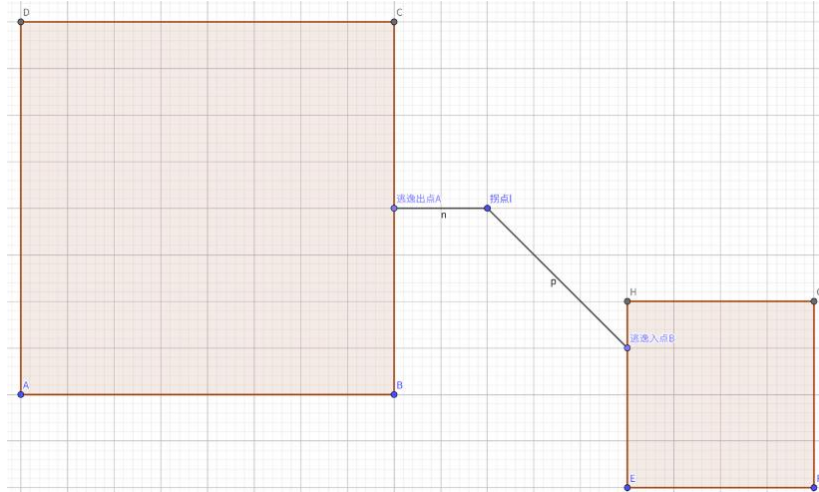


图 11: 一个拐点情况

第三类为有两个拐点情况。为了尽可能减少布线长度，出线和入线都与边界呈现 45 度角，其余影响拐点位置的因素只有逃逸点所在的逃逸边界方位，故共有  $4 \times 4 = 16$  种可能性，其中一种情况如图 12 所示。值得注意的是，当多条线需要两拐点平行布置时，如果不对拐点位置加以区分，就会在两个拐点产生的线段上重合。为了避免这种情况，本工作统计两拐点线网数目  $n$ ，同时计算拐点坐标让每个拐点之间都有相同的间隔，设定为 2。图 X 所示的拐点坐标计算公式为  $I(A.x + 2n, A.y - 2n), J(A.x + 2n, B.y + B.x - I.x)$

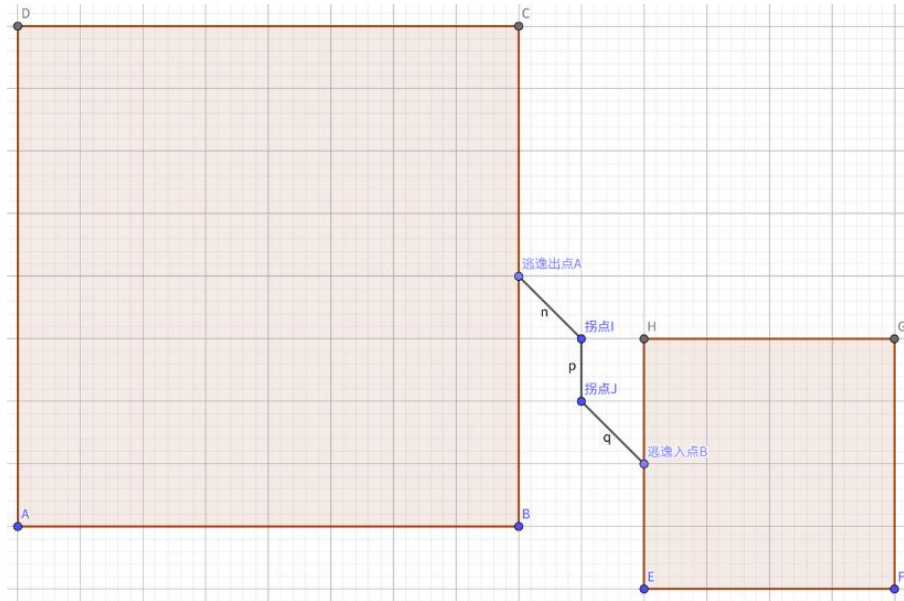


图 12: 两个拐点情况

将所有 52 种情况都写入程序即可以  $O(n)$  的时间复杂度对所有引脚完成区域布线。

3. 实验与讨论

3.1 实验设置

本工作在搭载 Intel(R) Xeon(R) CPU E5-2630 v3 @ 2.40GHz 的机器上，对于本文提出的算法进行了实验。其中，实验设备的内存容量为 528282624KB。实验中采用 4 线程并行配置。

实验中使用的布线测例来自于 2025 年 openDACS 开源 EDA 与芯片大赛中赛题 4 提供的竞赛基准数据集。该数据集中总共包含 4 个公开测试用例，但是，由于部分测例存在线网数目过多、网格矩阵规模过大、每个总线内引脚分布过于分散的问题，与本文提出的方法不适配，因此本文提出的方法在求解此类测例时效率过低，求解时间超过 2 小时。因此，本工作主要在该数据集中的 C2IO1 测例上对本文提出的算法进行实验。C2IO1 测例的相关参数如表 1 所示：

表 1 C2IO1 测例相关参数

| 参数名称   | 参数值  | 说明                      |
|--------|------|-------------------------|
| 单位网格边长 | 5    | 网格基本单元的边长，单位为微米         |
| 网格矩阵-长 | 4560 | 可布线区域的横向网格数目            |
| 网格矩阵-高 | 3840 | 可布线区域的纵向网格数目            |
| 线网数目   | 700  | 待布线的线网总数目               |
| 引脚数目   | 1400 | 待布线的引脚总数目，每个线网中包含 2 个引脚 |
| 芯粒数目   | 3    | 测例中参与布线的芯粒总数目           |

3.2 布线结果

在 C2IO1 测例上使用本文提出的算法进行布线，得到的结果如表 2 所示：

表 2 布线结果

| 布线指标   | 说明                           | 数值      |
|--------|------------------------------|---------|
| 线网布通率  | 布通线网数目/待布线线网总数目              | 700/700 |
| 布线总长度  | 全部线网的布线总长度，单位为网格数目           | 991606  |
| 平均布线长度 | 每个线网的平均布线长度，单位为网格数目          | 1416.58 |
| 布线总层数  | 包括 Top 层和 Bottom 层在内使用的布线总层数 | 3       |
| 求解总时间  | 单位为秒                         | 180.461 |

从上表中可以看出，本文提出的方法能够布通 C2IO1 测例中的全部待布线线网，求解总时间为 180.461 秒，约为 3 分钟。在布线过程中，本文提出的方法总共使用了 3 个布线层，即除了 Top 层和 Bottom 层，引入了一个额外的布线层；布线轨迹经过了 991606 个单位网格，即布线总长度为 4958030 微米；由于共计 700 个线网，故每个线网平均经过了 1416.58 个单位网格，即平均布线长度为 7082.9 微米。

为了展示本文提出算法的布线质量，本工作对布线结果进行了可视化，如图 13 所示：

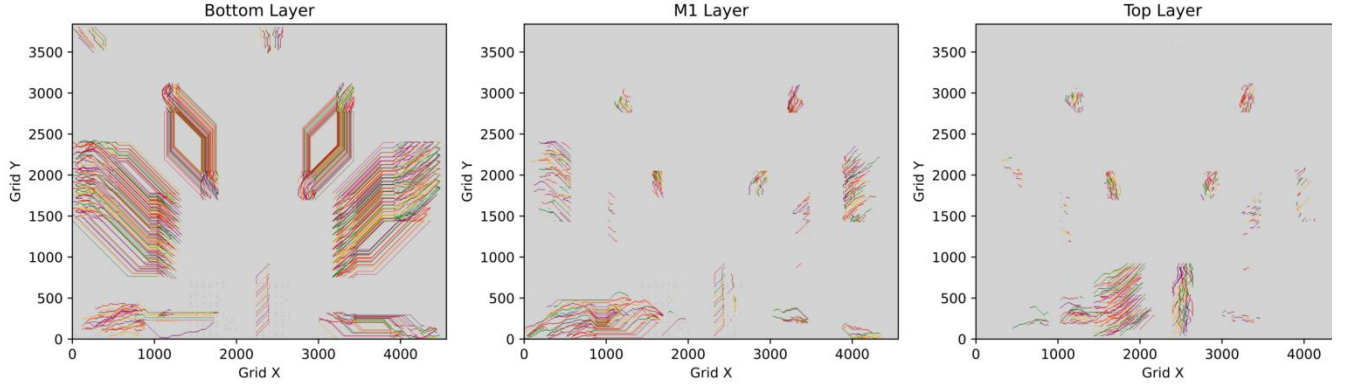


图 13. 布线结果可视化

在上图中，每个子图表示一个布线层。图中的不同颜色的折线表示不同线网的布线轨迹，其中，部分线网是跨层连通的。在某一个矩形区域内的不规则的布线轨迹是基于 A\*算法的逃逸布线的结果，比较规则的布线轨迹是基于模式的区域布线的结果。

综上所述，对于 C2IO1 测例，本文提出的算法能够在较短时间内布通测例中的全部待布线线网，同时具有较高的布线质量。

### 3.3 布线算法求解效率瓶颈分析

为了分析本文提出的布线算法的求解效率瓶颈，本工作统计了在求解 C2IO1 测例时各布线阶段耗时分布，除了输入输出以外，包括线网分类、总线布线、逃逸位置构建、逃逸布线、区域布线、重新布线，共计六个阶段，并绘制了柱状图，如图 14 所示：

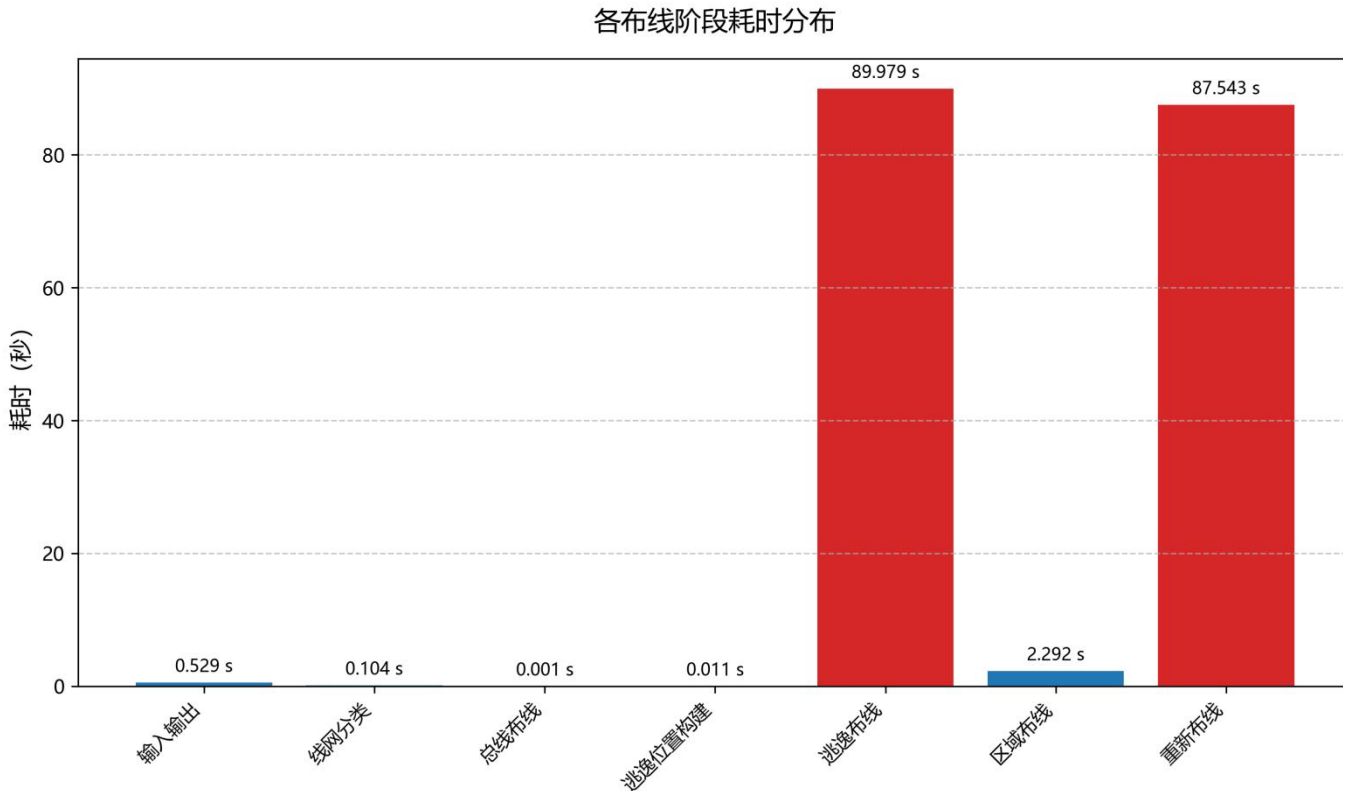


图 14. 布线算法各阶段耗时分布

从上图中可以看出，逃逸布线阶段和重新布线阶段是各布线阶段中耗时显著较大的两个阶段，这是因为逃逸布线和重新布线都是基于 A\*算法的，在求解过程中需要进行逐格探索，因此在网格规模较大时求解时间较长。对于其他布线阶段，线网分类和总线布线分别是基于 K-Means 聚类算法和图着色算法的，由于总线数目相对较少，因此求解效率较高；逃逸位置构建和区域布线均是基于启发式方法的，对于每个线网，分别均只需要求解两个坐标位置，即逃逸位置和拐点位置，因此求解效率也比较高。

综上所述，逃逸布线阶段和重新布线阶段是本文提出算法的求解效率瓶颈。

3.4 多线程并行优化性能验证

本工作使用了线程池的方式实现了多线程并行，并在实验过程中采用 4 线程并行配置。为了对多线程并行优化性能进行验证，本工作对比了使用单线程和 4 线程的算法分别求解 C2IO1 测例的实验结果，如表 3 所示：

表 3 单线程与 4 线程算法实验结果对比

| 布线指标   | 单线程     | 4 线程    |
|--------|---------|---------|
| 线网布通率  | 700/700 | 700/700 |
| 布线总长度  | 991606  | 991606  |
| 平均布线长度 | 1416.58 | 1416.58 |
| 布线总层数  | 3       | 3       |
| 求解总时间  | 345.468 | 180.461 |

从上述结果可以看出，使用单线程的算法与使用 4 线程的算法均布通了 C2IO1 测例中的全部线网，且布线总层数与布线总长度完全相同。在求解总时间上，使用单线程的算法总共耗时 345.468 秒，而使用 4 线程的算法总共耗时 180.461 秒，约为使用单线程的算法总求解时间的 52.2%。综上所述，在使用多线程并行优化后，能够在完全不牺牲线网布通率和布线质量的基础上，大幅度提升求解效率。

同时，本工作对比了使用单线程和 4 线程的算法分别求解 C2IO1 测例时各布线阶段的耗时，如表 4 所示：

表 4 单线程与 4 线程算法各布线阶段耗时对比

| 布线阶段   | 单线程            | 4 线程          |
|--------|----------------|---------------|
| 输入输出   | 0.315          | 0.529         |
| 线网分类   | 0.106          | 0.104         |
| 总线布线   | 0.001          | 0.001         |
| 逃逸位置构建 | 0.011          | 0.011         |
| 逃逸布线   | <b>192.460</b> | <b>89.979</b> |
| 区域布线   | 1.772          | 2.292         |
| 重新布线   | <b>150.802</b> | <b>87.543</b> |

从上述结果可以看出，使用单线程和 4 线程的算法分别求解 C2IO1 测例时，各布线阶段的耗时的主要区别在于逃逸布线阶段和重新布线阶段。这是因为本文提出的算法的多线程并行优化是作用于逃逸布线阶段和重新布线阶段的，主要原因如下：首先，根据布线算法求解效率瓶颈分析的结果，逃逸布线阶段和重新布线阶段是本文提出的算法的性能瓶颈，因此，存在对其进行并行优化的需求；其次，逃逸布线阶段和重新布线阶段均是基于线网分类和总线布线的结果进行分组进行的，而线网分类和总线布线能够保证各组之间的布线互相不干扰，因此，在逃逸布线阶段和重新布线阶段的各组之间进行多线程并行是可行的。因此，本工作选择在逃逸布线阶段和重新布线阶段实现多线程并行优化。

同时，因为各组之间的布线是相互独立的，因此，在加入多线程优化之后，布线结果不受到影响，因此，使用单线程和多线程的布线算法在求解 C2IO1 测例时，线网布通率和布线质量均完全相同。

综上所述，经过多线程并行优化，逃逸布线阶段和重新布线阶段的求解效率得到了大幅度的提升。然而，受限于本工作的数据结构设计，在多线程并行过程中，需要不断对部分全局变量进行加锁，这限制了多线程并行的效果。因此，4 线程并行带来的性能效率优化仅为二倍左右。这是本工作的未来优化方向之一。

4. 总结

本文针对 2.5D 芯粒基板布线中“有限空间内海量信号无冲突路径规划”的核心问题，提出了一套基于分组逃逸的完整布线算法，系统性地实现了布线效率与质量的协同优化。算法设计上，核心创新在于构建了“分类 - 分配 - 布线 - 优化”的四阶段框架：首先通过改进的 K-Means 聚类算法，以“斜率初划分 + 引脚位置精划分”实现线网智能分组，仅需一个斜率阈值超参数即可完成高效分类，为并行布线奠定基础；随后将总线布线转化为图着色问题，结合回溯法与 DSATUR 算法实现布线层的最优分配，在避免资源冲突的同时最小化布线层数；接着利用引脚分布规律构建逃逸位置，通过二维 A\*算法完成精准逃逸布线，并设计 52 种模式化路径实现 O(n) 复杂度的区域布线，大幅降低布线冗余；最后针对布线失败场景引入三维 A\*重新布线，保障布线完整性。此外，算法通过多线程并行优化聚焦性能瓶颈，在逃逸布线与重新布线阶段实现高效并行，显著提升求解效率。

实验验证方面,在 C2IO1 测例中,算法实现了 100% 线网布通率,仅需 3 层布线资源,求解总时间约 180 秒,平均布线长度控制在合理范围,布线质量与效率表现优异。多线程对比实验表明,4 线程配置较单线程效率提升 47.8%,且完全不牺牲布线质量,验证了并行优化的有效性。同时,本文也明确了算法的现存瓶颈:逃逸布线与重新布线阶段因基于 A\* 算法的逐格探索,在大规模网格场景下耗时占比极高;多线程并行效果受限于全局变量加锁的设计,未能充分发挥多核优势。此外,算法当前对部分线网密集、引脚分散的测例适配性不足,求解效率较低。

未来优化可聚焦三方面:一是优化数据结构设计,减少多线程并行中的锁竞争,进一步提升并行加速比;二是改进 A\* 算法的启发式函数与搜索策略,降低逐格探索的时间开销;三是拓展算法适配性,针对复杂测例优化线网分类与资源分配逻辑,增强算法的通用性。

## 参考文献

- [1] D. Baraff, "Dynamic simulation of non-penetrating rigid bodies," *Computer Graphics*, 1990, 24(4): 205-212.
- [2] K. O. Bowman, D. M. Foster, "The graph coloring problem: a recursive approach," *Communications of the ACM*, 1963, 6(10): 651-652.
- [3] Daniel Brélaz, "New methods to color the vertices of a graph," *Communications of the ACM*, 1979, 22(4): 251-256.
- [4] Peter E. Hart, Nils J. Nilsson, Bertram Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Trans. Systems Science and Cybernetics*, 1968, 4(2): 100-107.